

Evidencia Generación y Limpieza de Gramática

Carlos Delgado Contreras

A01712819

Implementación de Métodos Computacionales

Índice

Descripción.....	3
Estructura del Lenguaje.....	3
Sintagma Imperativo.....	4
Cadena Preposicional.....	4
Oración Completa.....	4
Modelos.....	5
Propuesta de Gramatica Libre De contexto.....	6
Limpieza e Implementación.....	8
Evidencia Limpieza.....	9
Pruebas.....	17
Prueba Gramatica original Codex_Inpurus.py (18 Reglas).....	18
Prueba Gramatica Limpia Puritas_Grammaticae.py (15 Reglas).....	21
Gramática Rechazada y Excepciones Gramática Limpia (15 reglas).....	24
Análisis.....	26
Jerarquía de Chomsky.....	26
Nivel de Jerarquía Gramatical Codex Impurus (ambigua) VS Puritas Grammati (no ambigua).....	26
Referencias.....	28

Descripción

El lenguaje seleccionado para la generación y limpieza de una Gramática Libre de Contexto fue la madre de las lenguas romances: el Latin. Este lenguaje se originó en la región del Lacio alrededor del siglo VIII a.C. Fue el lenguaje de los Latinos y evolucionó hasta convertirse en la lengua oficial del imperio Romano. Fue gracias a que esta civilización acuñó al Latin como lenguaje que este pudo convertirse en la “madre” de todas las lenguas romances. La expansión del Imperio Romano logró que el Latin llegara a lugares nuevos, combinándose y evolucionando por separado. Dando lugar al nacimiento de futuros lenguajes utilizados hasta hoy en día como es el Español, Portugués, Francés, Italiano, entre otros.

Estructura del Lenguaje

El latín es, por naturaleza, una lengua altamente flexible. A diferencia de las lenguas romances modernas, su gramática no depende de la posición de las palabras, sino de sus terminaciones (casos). Esta característica presenta un escenario único y complejo para el diseño de una gramática libre de contexto para un parser tipo LL(1).

Debido a que el latín permite un orden de palabras casi aleatorio (orden libre), intentar construir una gramática universal que cubra toda la lengua resultaría en una explosión de ambigüedad inmanejable para un algoritmo predictivo. Por ello, este proyecto delimita su alcance a frases célebres y aforismos. Estas estructuras, aunque mantienen la esencia gramatical latina, se han centralizado en formas sintácticas estables a lo largo de los siglos, permitiéndonos modelar una lógica predecible sin perder la riqueza del idioma.

Para garantizar la viabilidad del parser, el planteamiento se rige por tres realidades gramaticales del latín:

La Sustantivación: La capacidad de convertir verbos (infinitivos) o frases completas en sujetos funcionales, lo que introduce riesgos de recursión indirecta.

La Elipsis del Verbo Sum: La omisión frecuente del verbo copulativo en sentencias breves, obligando al uso técnico y preciso de la cadena vacía (ϵ).

La Morfología sobre la Sintaxis: La necesidad de tratar a los terminales no solo como palabras, sino como portadores de una función específica (Sujeto, Objeto, Atributo) independientemente de su posición original.

Para abordar este reto, hemos identificado tres estructuras fundamentales que encapsulan la mayoría de los aforismos latinos y que sirven como base para nuestros módulos de producción:

Sintagma Imperativo

Estructuras de mando o ruego que conectan verbos en imperativo con objetos o adverbios

Age quod agis: "Haz lo que haces" (Haz bien lo que estás haciendo).

Festina lente: "Aresúrate despacio" (La prisa es enemiga de la perfección).

Divide et impera: "Divide y vencerás" (Estructura: verbo imperativo + conjunción + verbo imperativo)

Cadena Preposicional

Construcciones basadas en la relación entre preposiciones y sustantivos que pueden extenderse de forma recursiva. Esta estructura suele ser preposición + sustantivo o preposición + sustantivo + preposición + sustantivo.

1. In vino veritas: "En el vino está la verdad" (Estructura: PREP + SUST + SUST).
Nota: El verbo "sum" suele estar omitido (elipsis).
2. Ex nihilo nihil: "De la nada, nada sale" (Estructura: PREP + SUST + SUST).
3. Ab Urbe Condita: "Desde la fundación de la ciudad" (Estructura: PREP + SUST + verbo participio).

Oración Completa

Esta es la estructura oracional completa. La estructura lógica de Sujeto + Predicado, que permite el análisis de afirmaciones filosóficas complejas y la interconexión entre diversos componentes gramaticales.

1. Amor vincit omnia: "El amor lo vence todo" (Estructura: SUJ + V_TRANS + OBJ).
2. Errare humanum est: "Error es humano" (Estructura: SUJ_INF + ATRIB + V_COP).
3. Verba volant, scripta manent: "Las palabras vuelan, lo escrito permanece" (Estructura doble: SUJ + V_INT + SUJ + V_INT).

Modelos

En computación, una gramática es un conjunto de **Reglas Matemáticas** que definen cómo se puede construir o generar palabras, frases u oraciones válidas. En las computadoras, para poder ____ se utilizar diferentes agentes para el análisis tanto léxico como gramatical. El análisis léxico agrupa en unidades con significado llamadas tokens el código fuente. Estos tokens son enviados al análisis gramatical también conocido como Parser el cual basándose en una gramática de libre contexto analiza la cualidad gramatical de los tokens y decide si es gramaticalmente correcto o no. Juntos estos componentes del compilador tienen como objetivo transformar este código fuente crudo en una estructura que la computadora pueda entender y procesar, llamada Árbol de derivación que produce un Árbol de Sintaxis Abstracta (AST). Un Árbol de Derivación representa todas las reglas de la gramática que se aplicaron para validar la cadena/código. Sirve como una forma de demostrar que el código es gramaticalmente correcto según las reglas formales. Si este árbol de derivación se limpia y simplifica se obtiene un AST que solo guarda los datos importantes que realmente usa el compilador para generar código o evaluar expresiones. Para este proyecto estaremos generando una gramática libre de contexto para un parser tipo Left-to-Right Leftmost Derivation 1. Este Parser ,perteneciente a la familia de los Top-Down, cuenta con visión “periférica” de 1 token por lo que no puede “predecir” el siguiente token al que entrará. Haciéndolo muy vulnerable a la recursión izquierda.

Propuesta de Gramatica Libre De contexto

Con el objetivo de generar una gramática de libre contexto que logre identificar a los anteriores aforismos, frases y refranes en latín en un parser LL(1). Se propone la siguiente gramática libre de contexto dividida en los siguientes módulos:

La gramática se define formalmente con la cuádrupla $G = (V, \Sigma, P, S)$, donde:

Conjunto de Variables (V):

$V = \{S, Oracion, Frase_Imp, SP_Cadena, Suj, Pred, Obj, Atrib\}$

Conjunto de Terminales (Σ):

$\Sigma = \{v_imp, v_trans, v_int, v_cop, v_inf, sust, prep, conj, adv, \epsilon\}$

Símbolo Inicial (S):

$S = S$

Reglas de Producción (P):

MÓDULO 1: Frases Imperativas

$Frase_Imp \rightarrow Frase_Imp + conj + Frase_Imp$

$Frase_Imp \rightarrow Frase_Imp + conj + v_imp$

$Frase_Imp \rightarrow v_imp + sust$

$Frase_Imp \rightarrow v_imp + adv$

$Frase_Imp \rightarrow v_imp$

MÓDULO 2: Estructuras Preposicionales

$SP_Cadena \rightarrow SP_Cadena + prep + sust$

$SP_Cadena \rightarrow SP_Cadena + sust$

$SP_Cadena \rightarrow \epsilon$ (Elipsis)

MÓDULO 3: Oraciones Completas

$Oracion \rightarrow Oracion + Oracion$

Oracion -> Suj + Pred

Oracion -> Suj + v_int

Pred -> v_trans + Obj

Pred -> Atrib + v_cop

Pred -> v_cop + Atrib

MÓDULO 4: Definiciones de Función

Suj -> sust | v_inf | Oracion | Frase_Imp | SP_Cadena

Obj -> Suj

Atrib -> Suj

S -> Oracion | Frase_Imp | SP_Cadena

El Diccionario de elementos que se utiliza es el siguiente:

v_imp -> 'divide' | 'impera' | 'vince' | 'memento' | 'carpe'

v_trans -> 'vincit'

v_int -> 'volant'

v_cop -> 'est'

v_inf -> 'errare' | 'mori'

sust -> 'diem' | 'amor' | 'omnia' | 'verba' | 'humanum' | 'aspera' | 'astra'

prep -> 'in' | 'ex' | 'per' | 'ad'

conj -> 'et'

adv -> 'lente'

Limpieza e Implementación

Tal cual la gramática propuesta anteriormente no puede ser procesada por un parser tipo LL(1) como el que vamos a utilizar para validar la gramática de las frases. Esto es debido a que contiene una **recursividad izquierda** inmensa (5 de sus 18 reglas de producción cuentan con recursión izquierda) la cual genera loops infinitos. Además nos encontramos con Alta ambigüedad al tener prefijos idénticos en las reglas de producción. Haciendo que el LL(1) no sepa a donde ir cuando se tope con el mismo prefijo pero diferentes caminos.

Por ejemplo:

La regla de producción para la Frase Imperativa tiene ¡3 caminos diferentes y válidos a la vez!

1. Frase_Imp -> v_imp + sust
2. Frase_Imp -> v_imp + adv
3. Frase_Imp -> v_imp

Esto genera ambigüedad en el parser LL(1).

Para lograr “limpiar” la ambigüedad, loops infinitos y recursividad izquierda ejecutamos las siguientes estrategias de limpieza en el siguiente orden.

1. Estratificación Jerárquica Establecimos una jerarquía semántica rigurosa. En lugar de permitir que cualquier símbolo derivara en otro indistintamente, definimos niveles de precedencia: los bloques de tokens atómicos (como verbos y sustantivos) son la base sobre la cual se construyen los bloques lógicos generales (oraciones y frases preposicionales). Esto elimina la ambigüedad donde un sujeto podía ser una oración y una oración podía ser un sujeto.

Para lograr esto debemos de seguir la siguiente clasificación paso a paso:

1. Clasificación por "Cercanía al Token"

Divide los símbolos en tres estratos claros para evitar que el parser se pierda en ciclos:

Nivel Atómico (Terminales): Define las categorías atómicas (sustantivos, verbos, etc.).

Nivel Estructural (Bloques): Define los grupos funcionales (Sujeto, Predicado, Frase Preposicional). Estos derivan en los atómicos.

Nivel Global (Raíz): La estructura superior que une los bloques (ej. S -> Oración).

Una vez identificado estos grupos podemos pasar a aplicar la lógica de Eliminación de la Recursión Izquierda

2. Eliminación de la Recursión Izquierda Detectamos los puntos donde la gramática se bloqueaba por auto-referencia inmediata, un error que provocaba que el parser entrara en un bucle infinito al intentar expandir una regla antes de procesar cualquier entrada. Para resolverlo, reescribimos las producciones utilizando la siguiente identidad algebraica:

$A \rightarrow Aa|B$ (es reemplazado por la dupla de abajo)

$A \rightarrow BA'$

$A' \rightarrow aA'|\epsilon$

Una vez aplicado este algoritmo en todos los casos de recursión izquierda, se pasa a factorizar por izquierda.

3. Factorización por Prefijo (Determinismo) La ambigüedad ocurría porque múltiples reglas compartían el mismo prefijo, dejando al parser "indeciso" ante el primer token recibido. Aplicamos un proceso de factorización donde extrajimos el factor común hacia un nuevo símbolo no terminal. Esto obliga al parser a consumir el token inicial y postergar la decisión de qué regla aplicar hasta haber obtenido suficiente contexto, garantizando una ruta única de derivación. Utilizamos la siguiente formula algebraica:

$A \rightarrow aB_1|aB_2$ (es reemplazado por la dupla de abajo)

$A \rightarrow aA'$

$A' \rightarrow B_1|B_2$

Una vez finalizado la factorización, contamos con reglas de producción limpias para un LL(1)

Evidencia Limpieza

A continuación se presenta evidencia de la limpieza hecha a mano de las 18 reglas de producción originales aplicando los conceptos y estrategias explicadas anteriormente. Generando las nuevas reglas de producción sin ambigüedad ni recursión izquierda.

Estratificación

Proyecto Limpieza (Lima 6, UCI): Estratificación

• Estratificación

* Módulo 4: Definiciones de Bases

1. Suj $\rightarrow$ Sust + token (Base)
2. Suj $\rightarrow$ V-int + token (Base)
3. Suj $\rightarrow$ Sust + V-int + token + NT (comp)
4. Suj $\rightarrow$ Frase-imp + SP-Cadena + NT (comp)
5. Suj $\rightarrow$ Suj + Pred $\rightarrow$ NT + NT (comp)
6. Suj $\rightarrow$ SP-Cadena $\rightarrow$ NT (comp)
7. Obj $\rightarrow$ Suj
8. Atrib $\rightarrow$ Suj

Nivel Alto: Suj-Base $\rightarrow$ Sust | V-int

Nivel Medio: Suj-Base $\rightarrow$ Sust | V-int

Nivel Bajo: Suj $\rightarrow$ Suj-Base | Suj-Base

Módulo 4: Estratificación

1. Suj-Base $\rightarrow$ Sust | V-int
2. Suj-Comp $\rightarrow$ Frase-imp | SP-Cadena
3. Suj $\rightarrow$ Suj-Base | Suj-Comp
4. Obj $\rightarrow$ Suj
5. Atrib $\rightarrow$ Suj

* Nota

Oración $\rightarrow$ Suj + Pred

Oración $\rightarrow$ Suj + V-int

* Módulo 2: Frases Imperativas

1. Frase-imp $\rightarrow$ Frase-imp + conj + Frase-imp
2. Frase-imp $\rightarrow$ Frase-imp + conj + V-imp
3. Frase-imp $\rightarrow$ V-imp + Sust
4. Frase-imp $\rightarrow$ V-imp + adv
5. Frase-imp $\rightarrow$ V-imp

Propuesta estratificación

Imp-Base $\rightarrow$ V-imp | V-imp + adv | V-imp + Sust

Frase-imp $\rightarrow$ Frase-imp + conj + Frase-imp | Imp-Base

* Módulo 2: Cadenas Preposicionales

1. SP-Cadena $\rightarrow$ SP-Cadena + Prep + Sust
2. SP-Cadena $\rightarrow$ SP-Cadena + Sust
3. SP-Cadena $\rightarrow$ e

Propuesta estratificación

SP-Base $\rightarrow$ Prep + Sust | Sust

SP-Cadena $\rightarrow$ SP-Cadena + SP-Base | SP-Base | e

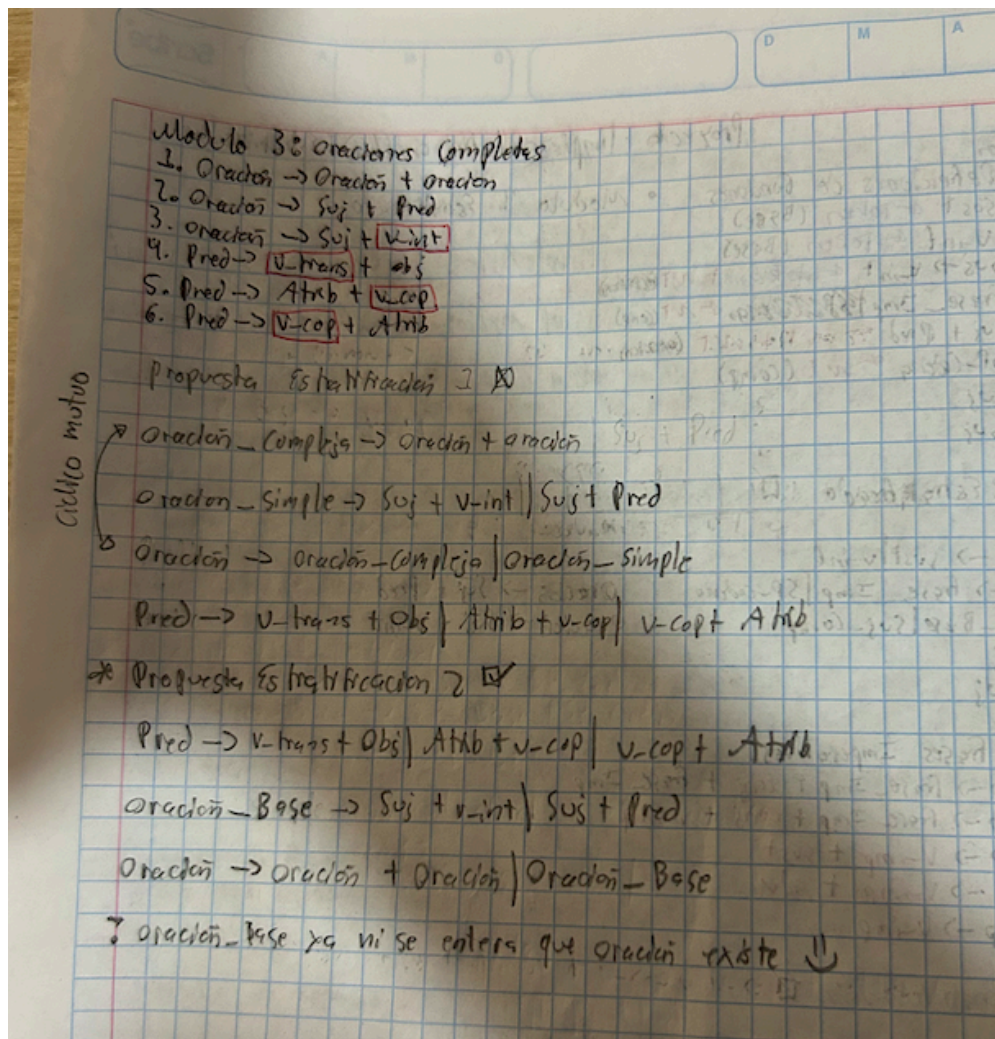
Derivación

Derivación

* Concepto $\rightarrow$ "Derivación" / "Sustitución"

SP-Base = Prep + Sust | Sust

Por eso lo podemos sustituir en SP-Cadena



Recursión Izquierda

Proyecto Limpieza Recursión Izquierda

* Módulo 2: Proposiciones

SP_Base $\rightarrow$ Prep + sust | Sust

SP_Cadena $\rightarrow$ SP_Cadena + SP_Base | SP_Base | ϵ

* Propuesta 2 Eliminación recursión izquierda

SP_Cadena $\rightarrow$ SP_Cadena + SP_Base | SP_Base | ϵ

SP_Cadena $\rightarrow$ SP_Base + SP_Cadena_Prima | ϵ

SP_Cadena_Prima $\rightarrow$ SP_Base + SP_Cadena_Prima | ϵ

* Módulo 2: Frases Imperativas

Imp_Base $\rightarrow$ V_imp | V_imp + adv | V_imp + sust

Frse_Imp $\rightarrow$ Frse_Imp + Frse_Imp | Imp_Base

Frse_Imp_Prima $\rightarrow$ Frse_Imp + Frse_Imp_Prima | ϵ

* Propuesta 3 Eliminación recursión izquierda

Frse_Imp $\rightarrow$ Imp_Base + Frse_Imp_Prima | ϵ

Frse_Imp_Prima $\rightarrow$ Frse_Imp + Frse_Imp_Prima | ϵ

* Módulo 3: Oraciones Completas

Pred $\rightarrow$ v-trans + obj | Atribut + v-cap | v-cap + Atribut

Oraden_Base $\rightarrow$ Suj + v-int | Suj + Pred

Oraden $\rightarrow$ Oraden + Oraden | Oraden_Base

Oraden $\rightarrow$ Oraden_Base + Oraden_Prima

Oraden_Prima $\rightarrow$ Oraden_Base + Oraden_Prima | ϵ

* Módulo 4

No tiene recursión izquierda

Regla de Oro: de algo hacia algo
 de lo más simple a lo más
 complejo o viceversa, pero
 lineal

Fórmula:
 $A \rightarrow PA'$
 $A' \rightarrow aA' | \epsilon$

Factorización Izquierda

$$A \rightarrow \alpha \beta_1 \mid \alpha \beta_2 \rightarrow A \rightarrow \alpha A' \\ A' \rightarrow \beta_1 \mid \beta_2$$
$$\text{Imp-Base} \rightarrow \underline{u_{\text{imp}}} | \underline{u_{\text{imp}}} + \frac{\text{adu}}{B_1} | \underline{v_{\text{imp}}} + \frac{\text{sust}}{B_2}$$
$$\text{Imp-Base} \rightarrow V_{\text{imp}} + \text{Imp-Base-prime}$$

Modulo 2: Preposiciones
 SP - Caten $\rightarrow$ SP - Base + SP - Cadena - Prima, le $\neq$ 3^o iguales

SP- Cadena-Prima $\rightarrow$ SP- Base + SP- Cadena-Prima

Tenemos redundancia estructural $\rightarrow$ la gramática es ultra predictiva

Aplicamos una fracción:

SP-Cadena $\rightarrow$ SP-Base + SP-Cadena e

pred $\rightarrow$ U-trans + obj / Atribut + U-cop / U-cop + Atribut

Oracion Base $\rightarrow$ $[S_{O_i}] + V_{int} + [S_{O_i}] + Pred$

Oracion $\rightarrow$ oracion base + oracion-prima

$\text{oracion} \rightarrow \text{oracion_base} + \text{oracion_primaria}$
 $\text{oracion_primaria} \rightarrow \text{oracion_base} + \text{oracion_primaria}$

Después de eso ambos se quedan

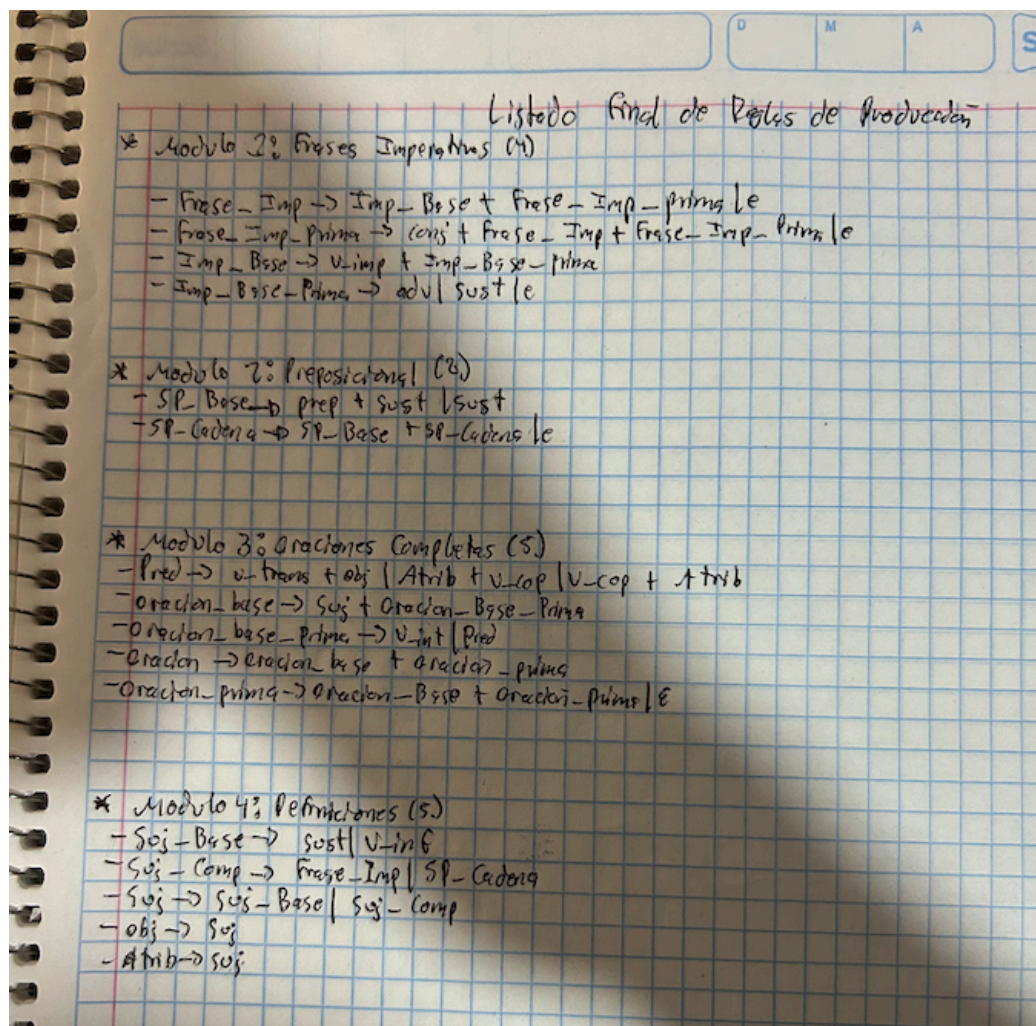
Propuesta de Limpieza Factorización

Oración Base $\rightarrow \frac{Suj + V_{int}}{a} \mid \frac{Suj + Pred}{a} \quad p_1 \quad p_2$

Ordnung - Base $\rightarrow$ Sus + Ordnung - Base - Prinzip

erachten - Base priming $\rightarrow V\text{-in} + | \text{Prod}$

Listado final de Reglas limpias:



La gramática limpia quedó de la siguiente manera (añadiendo frases extra):

Módulo 1: Frases Imperativas (4)

Frase_Imp $\rightarrow$ Imp_Base + Frase_Imp_Prima

Frase_Imp_Prima $\rightarrow$ conj + Frase_Imp + Frase_Imp_Prima | e

Imp_Base $\rightarrow$ v_imp + Imp_Base_Prima

Imp_Base_Prima $\rightarrow$ adv | sust | e

Módulo 2: Preposicional (2)

SP_Cadena $\rightarrow$ prep + sust + SP_Resto | e

SP_Resto $\rightarrow$ SP_Cadena | sust | e

Módulo 3: Oraciones Completas (5)

Pred $\rightarrow$ v_trans + Obj | Atrib + v_cop | v_cop + Atrib

Oracion_Base $\rightarrow$ Suj + Oracion_Base_Prima

Oracion_Base_Prima $\rightarrow$ v_int | Pred

Oracion $\rightarrow$ Oracion_Base + Oracion_Prima

Oracion_Prima -> Oracion_Base + Oracion_Prima | e

Modulo 4: Definiciones (5)

Suj -> sust + Suj_Tail

Suj -> v_inf

Suj -> Frase_Imp

Suj_Tail -> SP_Cadena

Obj -> Suj

Atrib -> Suj

Palabras:

Veni et vince (Verbo solo + Conjunción + Verbo solo)

Memento diem (Verbo + Sustantivo)

In vino veritas (prep + sust + sust)

Ex nihilo (prep + sust)

Astra volant: (Sustantivo + Verbo Intransitivo)

Amor vincit: (Sustantivo + Verbo Transitivo + Objeto Elíptico)

Terminales:

v_imp: Verbos en imperativo (Carpe, Divide, Memento).

sust: Sustantivos o nombres (Diem, Amor, Astra).

prep: Preposiciones (Per, Ad, In, Ex).

v_trans: Verbos transitivos (Vincit).

v_int: Verbos intransitivos (Volant, Manent).

v_cop: Verbo copulativo (Est).

v_inf: Verbos en infinitivo que actúan como sujeto (Errare).

conj: Conjunciones coordinantes (Et).

adv: Adverbios (Lente).

e (E): El símbolo de cadena vacía (elipsis).

No Terminales:

Módulo 1: Imperativos

- **Frase_Imp**: La estructura imperativa completa.
- **Frase_Imp_Prima**: La "cola" para permitir múltiples imperativos con conj.
- **Imp_Base**: Un bloque imperativo simple.
- **Imp_Base_Prima**: Factorización para decidir si sigue un sust o adv.

Módulo 2: Preposicionales

- **SP_Cadena**: Una sucesión de uno o más bloques preposicionales.
- **SP_Base**: Un solo bloque (*prep + sust* o solo *sust*).

Módulo 3: Oraciones

- **Oracion:** El símbolo inicial del sistema.
- **Oracion_Prima:** Permite encadenar varias oraciones (*Oracion + Oracion*).
- **Oracion_Base:** Estructura de una oración simple (*Sujeto + Predicado*).
- **Oracion_Base_Prima:** Factorización para decidir si sigue un *v_int* o un *Pred*.
- **Pred:** El predicado (Verbo transitivo/copulativo y sus complementos).

Módulo 4: Definiciones de Función

- **Suj:** El concepto abstracto de Sujeto.
- **Suj_Base:** La forma más simple de sujeto (*sust* o *v_inf*).
- **Suj_Comp:** Sujetos complejos (frases sustantivadas).
- **Obj:** El objeto directo.
- **Atrib:** El atributo del sujeto.

Pruebas

A continuación se presenta las pruebas ejecutadas en el script de python utilizando la librería nltk:

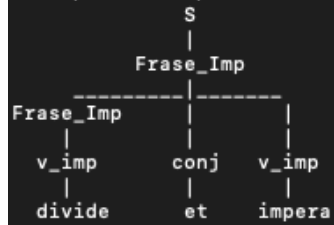
Prueba Gramatica original Codex_Inpurus.py (18 Reglas)

--- DEMOSTRACIÓN DE AMBIGÜEDAD Y RECURSIÓN IZQUIERDA ---

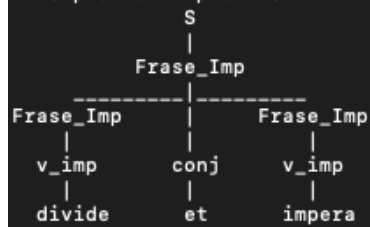
ANALIZANDO: 'DIVIDE ET IMPERA'

Ambigüedad Detectada: ¡Se encontraron 2 interpretaciones para esta frase!

Interpretación posible #1:

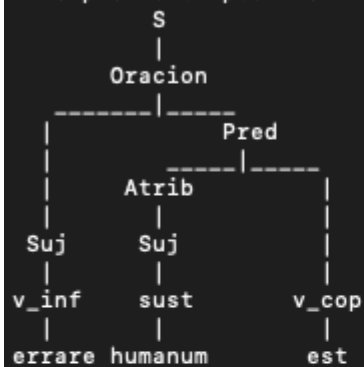


Interpretación posible #2:

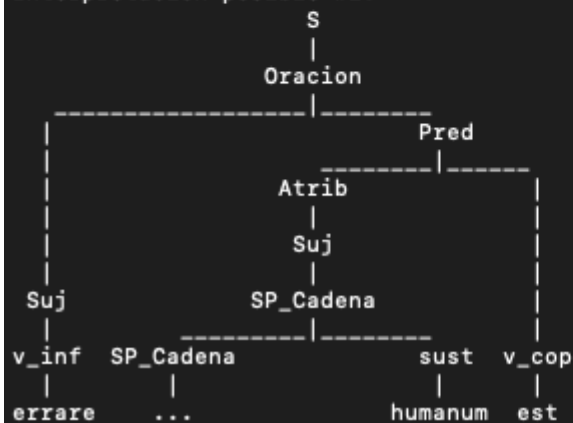


ANALIZANDO: 'ERRARE HUMANUM EST'
 Ambigüedad Detectada: ¡Se encontraron 2 interpretaciones para esta frase!

Interpretación posible #1:

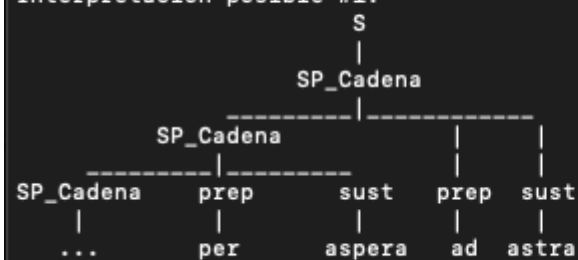


Interpretación posible #2:



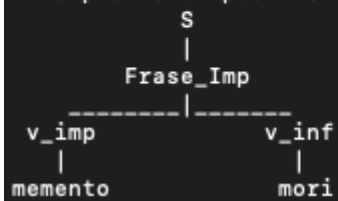
ANALIZANDO: 'PER ASPERA AD ASTRA'
 Ambigüedad Detectada: ¡Se encontraron 1 interpretaciones para esta frase!

Interpretación posible #1:



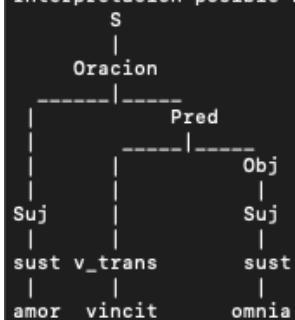
ANALIZANDO: 'MEMENTO MORI'
Ambigüedad Detectada: ¡Se encontraron 1 interpretaciones para esta frase!

Interpretación posible #1:

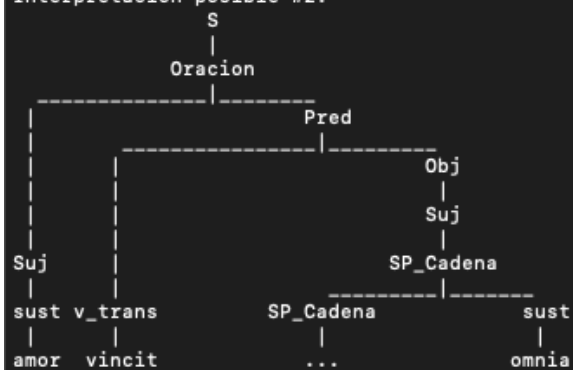


ANALIZANDO: 'AMOR VINCIT OMNIA'
Ambigüedad Detectada: ¡Se encontraron 4 interpretaciones para esta frase!

Interpretación posible #1:

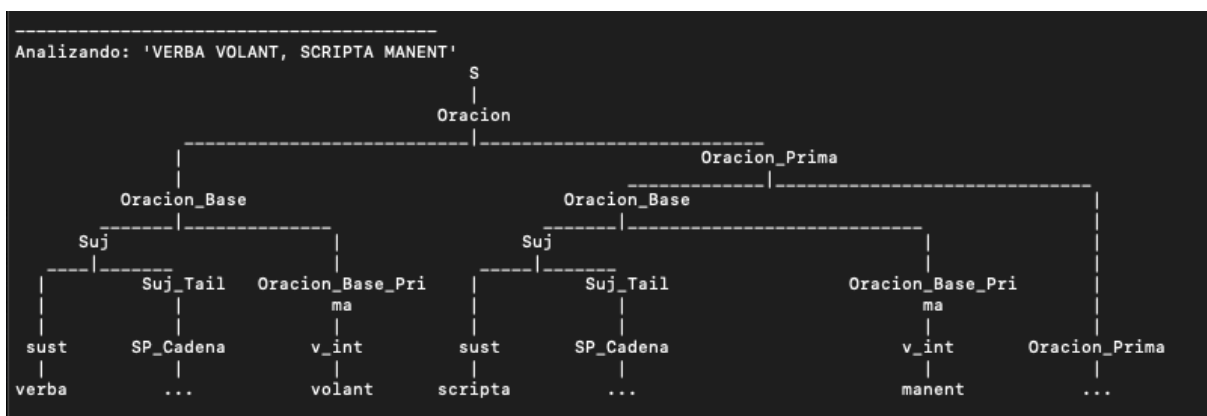
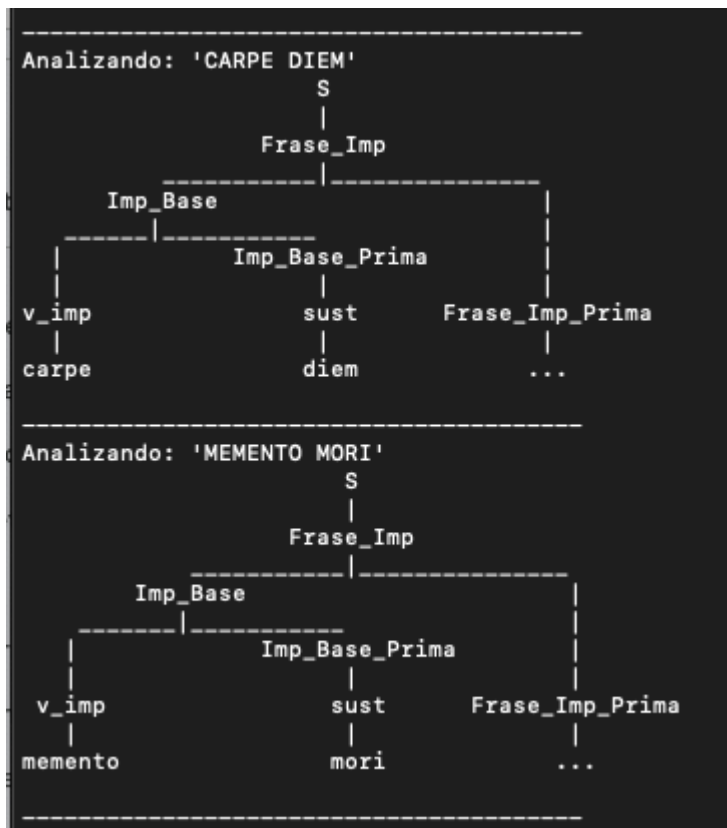


Interpretación posible #2:

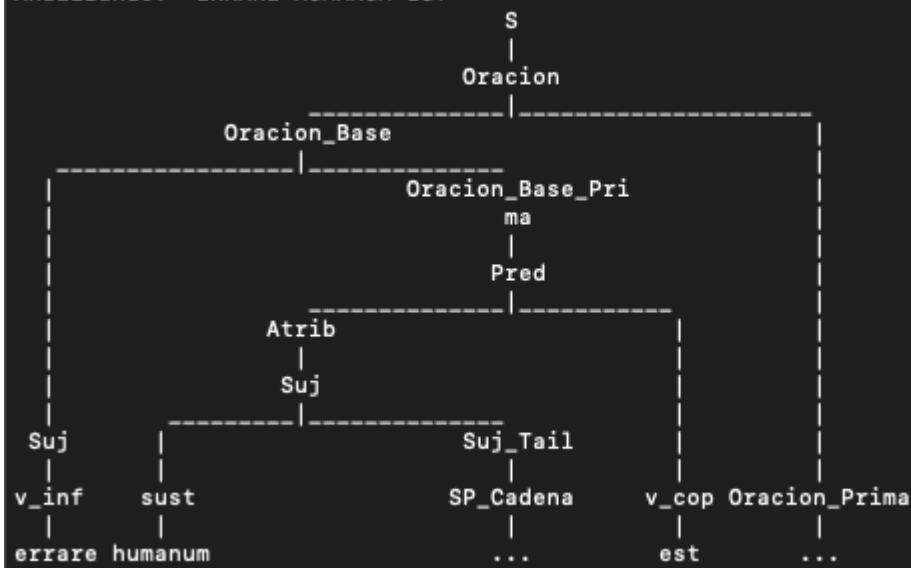


2 árboles más que no tienen sentido.

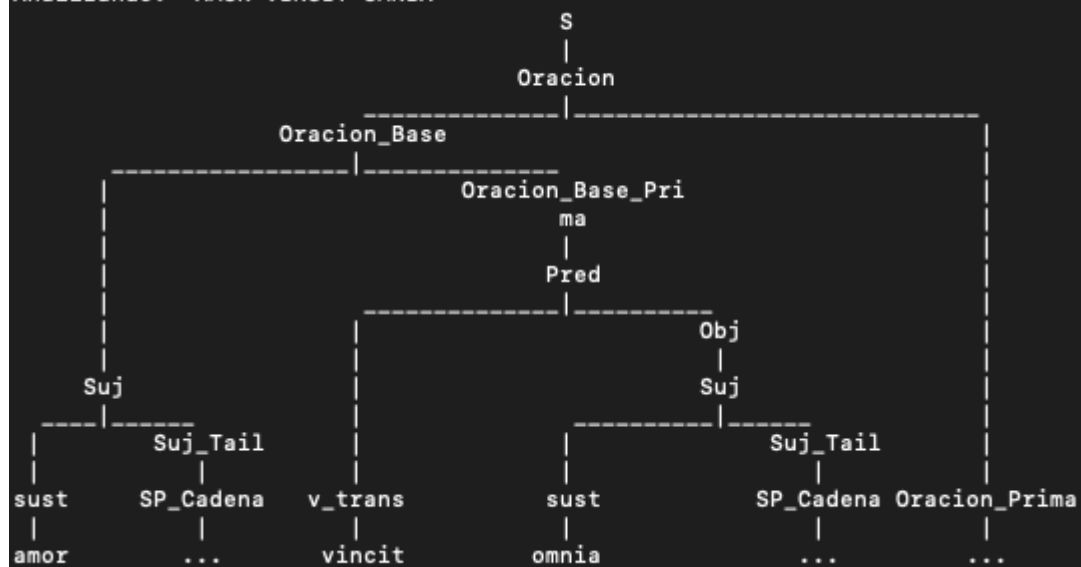
Prueba Gramatica Limpia Puritas_Grammaticae.py (15 Reglas)



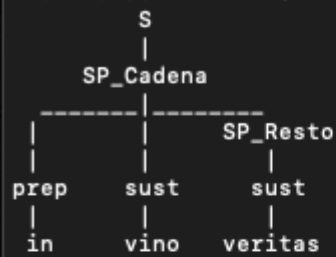
Analizando: 'ERRARE HUMANUM EST'



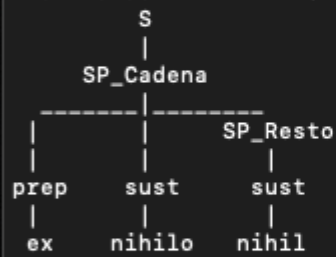
Analizando: 'AMOR VINCIT OMNIA'



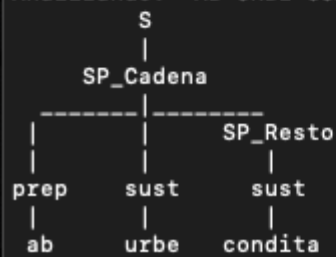
Analizando: 'IN VINO VERITAS'

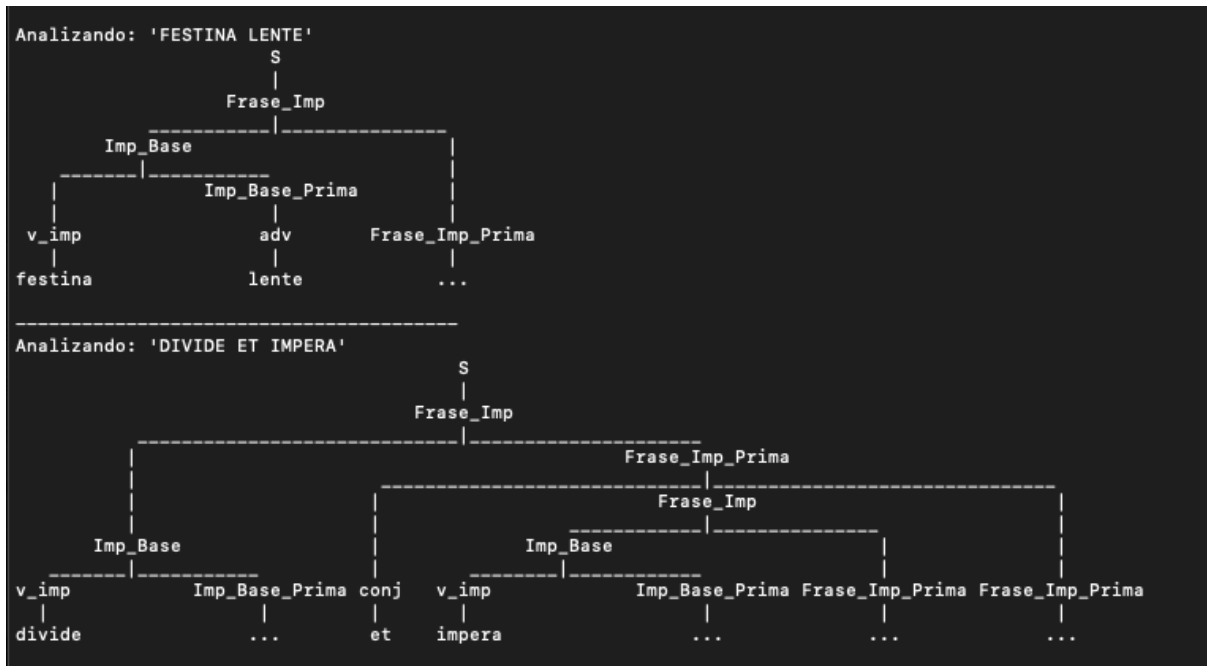


Analizando: 'EX NIHILO NIHIL'



Analizando: 'AB URBE CONDITA'





Gramática Rechazada y Excepciones Gramática Limpia (15 reglas)

```

Analizando: 'AGE QUOD AGIS'
-> [RECHAZADO] Error sintáctico: La frase excede la gramática actual.

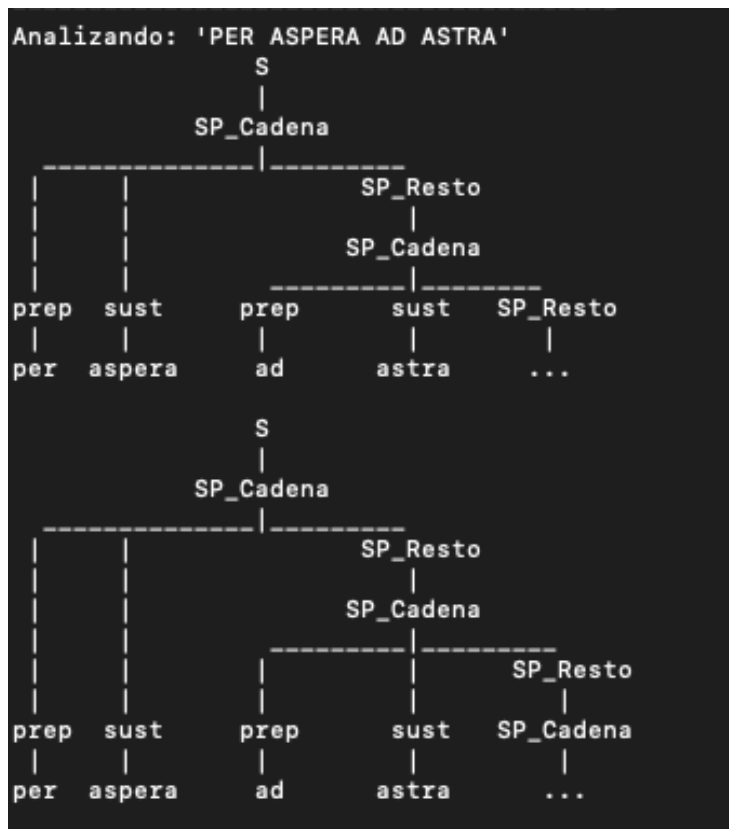
```

AGE QUOD AGIS es rechazado por la gramática en el módulo de Frases Imperativas:

Verbo Imperativo + (Un Solo Adverbio **O** Un Solo Sustantivo **O** Nada)

Cuando se detecta AGE (Verbo Imperativo) se mueve a la no terminal de Frase Imperativa esperando a continuación algún adverbio o sustantivo. Al recibir “Quod” el cual juega el papel de sustantivo se cierra la el bloque de Imp_Base_Prima esperando alguna conjunción para seguir. Agis no es una conjunción y el rol de sustantivo disponible ya lo tomo QUOD por lo que el parser no sabe que hacer con este Token y lo rechaza.

El problema de "Age quod agis" es que "quod agis" es una oración subordinada de relativo (una mini-oración actuando como objeto directo dentro de otra oración). La gramática actual no cuenta con un Módulo para poder manejar oraciones subordinadas. Tendríamos que diseñar una regla nueva que permitiera que un Objeto se convierta en una oración relativa. Por lo que llega al límite del Parser.



Durante la validación de la gramática Puritas, se observó un comportamiento contraintuitivo: la gramática refinada produjo dos derivaciones para la cadena "Per aspera ad astra", mientras que la gramática Codex Inpurus (original) solo produjo una. El fenómeno observado ocurre porque la gramática *Puritas* (15 reglas) posee dos reglas distintas que permiten llegar al estado de "cadena vacía" (ε) al final de la estructura. En teoría de gramáticas, si un símbolo puede derivar en el vacío a través de dos caminos sintácticos diferentes, el parser no puede determinar cuál de ellos representa la estructura lógica correcta. Este comportamiento señala un remanente de ambigüedad en la gramática que produce 2 caminos de “Salida” para “Per Aspera Ad Astra”

El Codex Inpurus (gramática sucia) exhibió un comportamiento determinista para esta frase específica debido a la Recursión Izquierda Directa. Al forzar que el símbolo crezca desde la izquierda, la gramática elimina los grados de libertad que tiene el parser. En la gramática sucia, la estructura está obligada de tal forma que solo existe un orden posible de aplicar las reglas; esto no significa que la gramática sea mejor, sino que su propia ineficiencia estructural la obliga a una única (aunque ineficiente) ruta de procesamiento.

Análisis

Jerarquía de Chomsky

Los lenguajes están divididos y seccionados en una organización que pondera su poder y entre más alto de nivel más complejo es al incluir las características de las anteriores. A esta clasificación se le conoce como la jerarquía de Chomsky. El Latin cuenta con la característica de la Sustantivación por lo que un sustantivo puede ser algo tan complejo como una oración o una frase. Esto hace que las reglas de producción tengan la siguiente característica:

Oración $\rightarrow$ Suj Pred

Suj $\rightarrow$ sust | v_inf | Oración

Para que esto sea posible necesitamos que el lenguaje pueda permitir que un símbolo pueda contenerse a sí mismo o a otro bloque lógico como por ejemplo una oración. El parser necesita ser posible de entrar y salir de la misma forma. Para esto hace falta tener un tipo de memoria como un Stack para “empujar” el estado actual y después sacarlo. Un autómata como un DFA o NFA, que son ciegos a la memoria, no podrían realizar esto y por ende el lenguaje Regular de nivel 3 no nos es útil. Debemos de subir un nivel más hacia los lenguajes libres de contexto. Estos lenguajes nos permiten tener “Memoria” en forma de un Push Down automático y a través de la libertad de poder poner cuál combinación de terminales y no terminales del lado derecho de la Regla de Producción:

$A \rightarrow r$ (donde r es cualquier combinación de terminales y no terminales)

Esto hace posible que nuestro lenguaje permite el anidamiento recursivo necesario para la sintaxis compleja del idioma.

Nivel de Jerarquía Gramatical Codex Impurus (ambigua) VS Puritas Grammati (no ambigua)

Tanto la gramática Puritas como el Codex Impurus están restringidas a producciones donde la parte izquierda es un único símbolo no terminal. Por lo tanto, bajo la clasificación estricta de Chomsky, ambas son Gramáticas Libres de Contexto (CFG). Lo que realmente las diferencia, y enormemente, es en la eficiencia algorítmica. La gramática 'Codex Impurus' presenta recursión izquierda y ambigüedad que impiden su procesamiento mediante autómatas de pila deterministas. En la práctica, esto obliga al sistema a recurrir a algoritmos de búsqueda exhaustiva (backtracking), elevando el costo computacional de la derivación. En contraste, la gramática 'Puritas' ha sido factorizada para ajustarse a un subconjunto de las gramáticas Tipo

2 conocido como LL(1), permitiendo un procesamiento determinista en tiempo lineal mucho más eficiente.

Cambios Post Entrega y Solución para Per Aspera Ad Astra

La producción presentada anteriormente contenía un cierto grado de ambigüedad en la frase “Per Aspera Ad Astra”. Esta frase produce 2 árboles en los que diferenciaba el modo de escape epsilon en la regla de producción de *SP_Resto* que de forma indirecta que ofrecía **dos caminos distintos** para poder llegar a un epsilon:

$SP_Cadena \rightarrow prep\ sust\ SP_Resto$

$SP_Resto \rightarrow SP_Cadena \mid sust \mid \epsilon$

El primer camino es la opción de entrar a una *SP_Cadena*, tomar de nuevo el *SP_Resto* y elegir salir con el Epsilon de *SP_Resto*. Como una ida a *SP_Cadena* y vuelta a Epsilon.

$SP_Resto \rightarrow SP_Cadena \rightarrow prep\ sust\ SP_Resto \rightarrow ad\ astra\ \epsilon$

En este último Epsilon tenemos la posibilidad de entrar de nuevo a *SP_Cadena* y hacer la vuelta de nuevo a Epsilon. Un recorrido innecesario pues terminamos de nuevo en Epsilon pero un camino posible que confunde al Parser.

La solución es eliminar la posibilidad de navegar ciclicamente entre *SP_Cadena* & *SP_Resto*. Si entramos en *SP_Resto* nos quedamos aquí y damos cuantas vueltas necesitemos.

Eliminamos la conexión $SP_Resto \rightarrow SP_Cadena$. Las reglas quedan de la siguiente manera:

$SP_Resto \rightarrow prep\ sust\ SP_Resto \leftarrow \text{continúa la cadena}$

$SP_Resto \rightarrow sust \leftarrow \text{cierra con sustantivo suelto}$

$SP_Resto \rightarrow \epsilon \leftarrow \text{cierra vacío}$

La toma de decisiones se queda en *SP_Resto* sin la posibilidad de hacer viajes redundantes entre reglas de producción. De esta forma se elimina la ambigüedad restante.

Referencias

Aho, A. V., Lam, M. S., Sethi, R., & Ullman, J. D. (2006). *Compilers: Principles, Techniques, and Tools* (2nd ed.). Pearson Education.